Lab 14 — Measure, Version and Write the Team Playbook

Claude Cowork for Digital Marketing · Day 2 · Use Claude Code

> **USE: CLAUDE CODE** — Code mode (the </> button) in the Claude desktop app, in your horizon-studio folder

Course: Claude Cowork for Digital Marketing (TGS-2023018659)

Day 2 · Topic 3 · about 40 minutes · slides 160–166

Surface: Claude Code (subagents)

Features: compare agent patterns · the roster: instructions, skills, connectors · approval matrix · responsible-AI controls · version history · a change proposal and a roll-back plan

> **Copy the prompts:** https://tertiarycourses.github.io/TGS-2023018659-Claude-Cowork-for-Digital-Marketing/prompts/claude/#lab-14 — click **Copy** next to each prompt, then paste it into the chat.

The story so far

Eight weeks in. Rachel asks which ways of running agents earned their tokens, and how the team runs safely after you leave.

Your goal

Rachel asks: which patterns earned their tokens, and how does the team run safely after you leave? Answer with numbers, then with a playbook a new hire can follow — then change the team safely: version it, propose one change, test it, keep a way back.

You'll build

the agent-pattern comparison, the team playbook, a change proposal, and versions 1.0 and 1.1 of the team — shown as artifacts, with a Word copy of each

What is in this folder

> **.md or .pdf?** A `.md` file (Markdown) is plain text that Claude reads. Every `.md` here has a `.pdf` twin with the same name — open the PDF to read it yourself.

- `assets/run-log.csv`
- `assets/playbook-outline.md`
- `assets/responsible-ai-checklist.md`
- `prompts.md` / `prompts.pdf` — every prompt, ready to paste
- `evidence/checklist.md` / `checklist.pdf` — what to capture as proof

Step by step

1. **New session** — Left sidebar: click + next to horizon-studio. Click the session title at the top and rename it "Lab 14". Run all of this lab's prompts in it.
2. **Check the data** — The assets are already in data/. Open one and skim it.
3. **Compare the patterns** — Copy and paste Prompt A.
4. **Write the playbook** — After the Lead shows the comparison, copy and paste Prompt B.
5. **Review it as Rachel** — Add one rule of your own and say why.
6. **Propose and version a change** — After you have added your rule, copy and paste Prompt C. Claude saves version 1.0, makes one change, tests it and saves 1.1.
7. **Compare the versions** — Read what changed between 1.0 and 1.1. If a check failed, ask: "Roll back to version 1.0."

The prompts

PROMPT A — Claude Code: compare

> Use the growth-analyst subagent. From
> data/run-log.csv compare the patterns we used —
> one agent, subagents, agent teams and the scheduled task
> — on minutes, tokens, defects caught before a person
> and human edits. Which pattern pays off for which
> job? Save reports/agent-comparison.md.
> Publish it as an artifact: one chart per measure,
> comparing the patterns.
> Write for a business owner, in plain English: no
> codes, IDs, file names or line numbers; start with
> what it means and what I do next. Save a Word copy
> (.docx) of each result.

PROMPT B — Claude Code: the playbook

> Write strategy/team-playbook.md with
> data/playbook-outline.md. Include the roster: for
> each agent its instructions file in .claude/agents/, its
> skills, connectors, model and what it may never do;
> the approval matrix by risk; the rules in
> data/responsible-ai-checklist.md; and what to do when
> a wrong post goes out. Then run fin-compliance on
> it.
> Publish the playbook as an artifact: the roster as
> cards, the approval matrix as a table.
> Write for a business owner, in plain English: no
> codes, IDs, file names or line numbers; start with
> what it means and what I do next. Save a Word copy
> (.docx) of each result.

PROMPT C — Claude Code: version and propose a change

> Keep a version history of this folder (use git;
> set it up if it is not there yet). Save everything
> as version 1.0 with the note "The team as built in
> class".
> From reports/agent-comparison.md and
> data/run-log.csv, propose ONE change to how the team
> works. Write review/change-proposal.md: what we saw,
> the change, the files it touches, how we test it,
> the risk, who approves, and how we roll back.
> Make the change, test it on one recent piece with
> fact-check and fin-compliance, and save version 1.1
> with a one-line note. Show me what changed between
> 1.0 and 1.1, and how to go back to 1.0.
> Publish the proposal and the version history as an
> artifact.
> Write for a business owner, in plain English: no
> codes, IDs, file names or line numbers; start with
> what it means and what I do next. Save a Word copy
> (.docx) of each result.

Ask about the result

Once a result opens, type any of these in the chat to get its meaning. Ask until it makes sense to you — then decide.

- "In one line each: when should we use one agent, subagents and an agent team?"
- "Who must approve a post that has a CPF figure in it?"
- "A wrong post went out. What are the first three things we do?"
- *After Prompt C:* "In plain English, what changed in version 1.1, and how would we undo it?"

Check your work

- ☐ The comparison shows the one-agent baseline caught 0 defects.
- ☐ It says which pattern suits which job, with numbers.
- ☐ The roster lists all six agents with skills and connectors.
- ☐ The approval matrix says who approves CPF and tax figures.
- ☐ It has an incident procedure, and your own rule.
- ☐ Versions 1.0 and 1.1 are saved, each with a note.
- ☐ The change proposal has the evidence, the test, the risk and the roll-back.

If it goes wrong

- **The playbook is generic** — Say: "use our actual agents, files and numbers".
- **Version history is not available** — Ask: "Keep versions as dated copies in _versions/ with a CHANGELOG.md instead." The change proposal stays the same.

Stretch

- Turn the playbook into a skill, team-playbook, for the Lead.

> **Why it matters:** Teams cost more tokens. The run log shows what you buy with them: defects caught before a person, and fewer edits.

Next

Lab 15 — Agent Teams: A Strategy Debate and a QA Loop. Keep what you built — the next lab starts from it.

Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password, token or client data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without a person approving that exact version.